



Implementing Security Measures

Submitted by

Jannat Yousaf

DHC-826

Acknowledgement

I would like to express my sincere gratitude to my internship team leader for their continuous guidance and support throughout this project. I also appreciate the learning opportunity provided during this internship, which enabled me to enhance my technical and problem-solving skills. This experience has been highly beneficial for my professional growth.

Table of Content

Implementing Security Measures-----	3
Implementing Security Measures in Web Application -----	3
Introduction-----	3
Security Measures Implemented-----	3
Challenges Faced -----	4
Tasks Completed-----	4
Solutions Implemented-----	5
Conclusion -----	5

Implementing Security Measures

Implementing Security Measures in Web Application

Introduction

In this project, a secure web application was developed using Node.js and Express.js to understand and implement fundamental security practices. The main objective was to protect user data and prevent common web vulnerabilities such as unauthorized access and injection attacks.

Security Measures Implemented

1. Input Validation

User inputs were validated using the validator library to ensure only properly formatted data is accepted. For example, email validation was implemented to prevent invalid or malicious inputs.

```
const validator = require('validator');  
if (!validator.isEmail(email)) {  
  return res.status(400).send('Invalid email');  
}
```

2. Password Hashing

User passwords were secured using bcrypt hashing. Instead of storing plain text passwords, hashed passwords were stored in memory, ensuring data protection even if the system is compromised.

```
const bcrypt = require('bcrypt');  
const hashedPassword = await bcrypt.hash(password, 10);  
  
password comparison:  
const isMatch = await bcrypt.compare(password, user.password);  
if (!isMatch) {  
  return res.status(400).send('Invalid credentials');  
}
```

3. Authentication using JWT

JSON Web Tokens (JWT) were implemented for secure authentication. Upon successful login, a token is generated and sent to the client, which is then used to access protected routes such as the dashboard.

```
const jwt = require('jsonwebtoken');  
const token = jwt.sign({ email: user.email }, 'secretkey', {  
  expiresIn: '1h'  
});
```

4. Protected Routes

Access to the dashboard was restricted using token-based authentication. Unauthorized users are redirected to the login page, ensuring secure access control.

```
const token = req.headers['authorization'];
if (!token) {
  return res.status(401).send('Access denied');
}
jwt.verify(token, 'secretkey', (err, decoded) => {
  if (err) {
    return res.status(403).send('Invalid token');
  }
  res.send('Welcome to dashboard');
});
```

5. Secure HTTP Headers (Helmet)

Helmet middleware was used to set secure HTTP headers. This helped protect the application from common attacks such as Cross-Site Scripting (XSS) and clickjacking.

```
const helmet = require('helmet');
app.use(helmet());
```

6. Content Security Policy (CSP)

CSP was enforced using Helmet, which blocked inline JavaScript execution. This required restructuring the frontend by moving inline scripts to external JavaScript files, improving overall security.

7. Logout Functionality

A logout mechanism was implemented by removing the authentication token from local storage, ensuring users can securely end their session.

Challenges Faced

- Handling undefined request body due to missing middleware
- Resolving internal server errors during API calls
- Understanding and fixing Content Security Policy (CSP) issues
- Debugging frontend-backend communication

Tasks Completed

- Developed a secure user registration and login system

- Implemented password hashing using bcrypt
- Integrated JWT-based authentication
- Created a protected dashboard accessible only to authenticated users
- Applied input validation using validator
- Secured application using Helmet middleware
- Designed a simple frontend with registration, login, and dashboard pages

Solutions Implemented

- Added `express.json()` middleware
- Used Postman for API testing
- Replaced inline JavaScript with external JS files
- Followed secure coding practices for frontend and backend

Conclusion

This project provided practical experience in implementing web security measures. It enhanced understanding of authentication, data protection, and secure coding practices, which are essential for modern web development.